

```
const cityCoordinates = {
  sochi: [43.585525, 39.723062],
  moscow: [55.751244, 37.618423],
  kazan: [55.796127, 49.106405],
  spb: [59.93428, 30.335098]
};

const cityNames = {
  sochi: "Сочи",
  moscow: "Москва",
  kazan: "Казань",
  spb: "Санкт-Петербург"
};

const userTypeLabels = {
  default: "по умолчанию",
  wheelchair: "человек на коляске",
  stroller: "родитель с коляской",
  senior: "пожилой человек"
};

let map;
let pointCollection;
let multiRoute;

function addAccessibilityPoints(center) {
  const lat = center[0];
  const lng = center[1];

  const points = [
    {
      coordinates: [lat + 0.004, lng - 0.006],
      preset: "islands#greenCircleDotIcon",
      title: "Доступный переход",
      text: "Съезд найден с обеих сторон дороги."
    },
    {
      coordinates: [lat + 0.001, lng - 0.001],
      preset: "islands#orangeCircleDotIcon",
      title: "Требуется проверки",
      text: "Рекомендуется уточнить состояние покрытия и бордюров."
    },
    {
      coordinates: [lat - 0.002, lng + 0.005],
```

```

        preset: "islands#redCircleDotIcon",
        title: "Возможное препятствие",
        text: "Возможен высокий бордюр или неровное покрытие."
    }
];

points.forEach(function (item) {
    pointCollection.add(
        new ymaps.Placemark(
            item.coordinates,
            {
                balloonContentHeader: item.title,
                balloonContentBody: item.text
            },
            {
                preset: item.preset
            }
        )
    );
});
}

function initMap() {
    map = new ymaps.Map("map", {
        center: cityCoordinates.sochi,
        zoom: 13,
        controls: ["zoomControl", "geolocationControl", "typeSelector"]
    });

    pointCollection = new ymaps.GeoObjectCollection();
    map.geoObjects.add(pointCollection);
    addAccessibilityPoints(cityCoordinates.sochi);
}

function clearMapData() {
    pointCollection.removeAll();

    if (multiRoute) {
        map.geoObjects.remove(multiRoute);
        multiRoute = null;
    }
}

function prepareAddress(address, cityName) {
    const normalizedAddress = address.trim();
    const normalizedCity = cityName.toLowerCase();

```

```

    if (normalizedAddress.toLowerCase().startsWith(normalizedCity)) {
        return normalizedAddress;
    }

    return cityName + ", " + normalizedAddress;
}

function buildRoute(fromAddress, toAddress, city, userType) {
    const message = document.getElementById("routeMessage");
    const resultBlock = document.getElementById("routeResult");
    const description = document.getElementById("resultDescription");
    const cityName = cityNames[city];

    clearMapData();
    resultBlock.hidden = true;
    message.textContent = "Строю пешеходный маршрут...";

    const startPoint = prepareAddress(fromAddress, cityName);
    const endPoint = prepareAddress(toAddress, cityName);

    multiRoute = new ymaps.multiRouter.MultiRoute(
        {
            referencePoints: [startPoint, endPoint],
            params: {
                routingMode: "pedestrian",
                results: 1
            }
        },
        {
            boundsAutoApply: true,
            wayPointVisible: true,
            viaPointVisible: false,
            routeActiveStrokeColor: "#a8677a",
            routeActiveStrokeWidth: 6,
            routeInactiveStrokeColor: "#e3cdd3",
            routeInactiveStrokeWidth: 4
        }
    );

    map.geoObjects.add(multiRoute);
    addAccessibilityPoints(cityCoordinates[city]);

    multiRoute.model.events.add("requestsuccess", function () {
        resultBlock.hidden = false;
        description.textContent =
            "Маршрут построен через Яндекс Карты по пешеходным дорожкам. Метки на карте
            (userTypeLabels[userType] || userTypeLabels.default) + ".";
    });
}

```

```

        message.textContent =
            "Маршрут построен. Нажмите на цветные метки на карте, чтобы увидеть пояснен
    });

    multiRoute.model.events.add("requestfail", function (event) {
        console.error("Ошибка маршрутизатора Яндекс:", event);
        message.textContent =
            "Не удалось построить маршрут. Проверьте, что адреса существуют в выбранном
    });
}

const citySelect = document.getElementById("citySelect");
const cityStatus = document.getElementById("cityStatus");
const routeForm = document.getElementById("routeForm");
const routeMessage = document.getElementById("routeMessage");

citySelect.addEventListener("change", function () {
    const city = citySelect.value;

    cityStatus.textContent = cityNames[city];
    clearMapData();
    map.setCenter(cityCoordinates[city], 13);
    addAccessibilityPoints(cityCoordinates[city]);
    routeMessage.textContent =
        "Заполните поля «Откуда» и «Куда», затем нажмите «Построить маршрут».";
});

routeForm.addEventListener("submit", function (event) {
    event.preventDefault();

    const from = document.getElementById("fromInput").value.trim();
    const to = document.getElementById("toInput").value.trim();
    const city = citySelect.value;
    const userType = document.getElementById("userType").value;

    if (!from || !to) {
        routeMessage.textContent = "Заполните оба поля: «Откуда» и «Куда».";
        return;
    }

    buildRoute(from, to, city, userType);
});

const authModal = document.getElementById("authModal");
const openAuthButton = document.getElementById("openAuthButton");
const closeAuthButton = document.getElementById("closeAuthButton");

```

```
openAuthButton.addEventListener("click", function () {
    authModal.hidden = false;
});

closeAuthButton.addEventListener("click", function () {
    authModal.hidden = true;
});

authModal.addEventListener("click", function (event) {
    if (event.target === authModal) {
        authModal.hidden = true;
    }
});

document.getElementById("authForm").addEventListener("submit", function (event) {
    event.preventDefault();

    const name = document.getElementById("nameInput").value.trim();
    const email = document.getElementById("emailInput").value.trim();

    localStorage.setItem(
        "accessibleRouteAccount",
        JSON.stringify({ name: name, email: email })
    );

    document.getElementById("authMessage").textContent =
        "Учётная запись для " + name + " сохранена в этом браузере.";

    openAuthButton.textContent = name;
    event.target.reset();
});

try {
    const account = JSON.parse(localStorage.getItem("accessibleRouteAccount"));

    if (account && account.name) {
        openAuthButton.textContent = account.name;
    }
} catch (error) {
    localStorage.removeItem("accessibleRouteAccount");
}

ymaps.ready(initMap);
```